



5 Esercizi

Esercizi numerici di ricorsione

1. **Somma dei primi numeri dispari.** Scrivere una funzione ricorsiva che calcoli la somma dei primi n numeri dispari positivi.
Suggerimento (da considerare solo in caso di difficoltà): l'ultimo termine può essere espresso come $2n - 1$.
2. **Conteggio delle cifre di un numero.** Scrivere una funzione ricorsiva che, dato un numero intero positivo n , restituisca il numero di cifre che lo compongono.
Suggerimento (da considerare solo in caso di difficoltà): dividere n per 10 a ogni passo riduce di una cifra il numero.
3. **Somma delle cifre di un numero.** Scrivere una funzione ricorsiva che calcoli la somma delle cifre di un numero intero positivo n .
Suggerimento (da considerare solo in caso di difficoltà): l'ultima cifra di n può essere ottenuta con $n \bmod 10$.
4. **Massimo comune divisore (MCD).** Scrivere una funzione ricorsiva che calcoli il massimo comune divisore tra due numeri interi positivi a e b .
Suggerimento (da considerare solo in caso di difficoltà): utilizzare la relazione $\text{MCD}(a, b) = \text{MCD}(b, a \bmod b)$.
5. **Conteggio dei divisori di un numero.** Scrivere una funzione ricorsiva che conti quanti interi positivi dividono esattamente un dato numero n .
Suggerimento (da considerare solo in caso di difficoltà): verificare se $n \bmod k = 0$ per un divisore candidato k decrescente.
6. **Prodotto dei primi numeri dispari.** Scrivere una funzione ricorsiva che calcoli il prodotto dei primi n numeri dispari positivi.
Suggerimento (da considerare solo in caso di difficoltà): l'ultimo termine della sequenza è $2n - 1$.

7. **Successione di Fibonacci.** Scrivere una funzione ricorsiva che calcoli il termine n -esimo della successione di Fibonacci.
Suggerimento (da considerare solo in caso di difficoltà): la successione è definita da $F(0) = 0$, $F(1) = 1$ e $F(n) = F(n-1) + F(n-2)$.

8. **Somma alternata dei primi n interi positivi.** Scrivere una funzione ricorsiva che calcoli la somma alternata dei primi n interi positivi:

$$S(n) = 1 - 2 + 3 - 4 + 5 - 6 + \dots \pm n.$$

Suggerimento (da considerare solo in caso di difficoltà): la regola può essere espressa come $S(n) = S(n-1) + (-1)^{n+1} \cdot n$.

Esercizi di ricorsione su array

1. **Conteggio dei valori negativi.** Scrivere una funzione ricorsiva che conti quanti elementi di un array A di lunghezza n sono minori di zero.
Suggerimento (da considerare solo in caso di difficoltà): sommare 1 se $A[i] < 0$ e procedere con l'array rimanente.
2. **Prodotto dei valori pari.** Scrivere una funzione ricorsiva che calcoli il prodotto di tutti gli elementi pari di un array A di lunghezza n . Se l'array non contiene elementi pari, la funzione deve restituire 1.
Suggerimento (da considerare solo in caso di difficoltà): verificare la condizione $A[i] \bmod 2 = 0$ prima di moltiplicare.
3. **Conteggio dei valori superiori alla media.** Scrivere una funzione ricorsiva che conti quanti elementi di un array A di lunghezza n hanno valore superiore alla media aritmetica dell'array stesso.
Suggerimento (da considerare solo in caso di difficoltà): calcolare prima la media con una funzione separata e utilizzarla come parametro nella chiamata ricorsiva principale.
4. **Differenza tra somma dei pari e somma dei dispari.** Scrivere una funzione ricorsiva che calcoli la differenza tra la somma degli elementi pari e la somma degli elementi dispari di un array A di lunghezza n .
Suggerimento (da considerare solo in caso di difficoltà): usare una ricorsione unica in cui ogni elemento contribuisce con segno positivo o negativo a seconda della sua parità.
5. **Verifica di simmetria.** Scrivere una funzione ricorsiva che determini se un array A di lunghezza n è simmetrico rispetto al suo centro, cioè se $A[i] = A[n-1-i]$ per ogni i . La funzione deve restituire **true** o **false**.
Suggerimento (da considerare solo in caso di difficoltà): confrontare gli elementi alle estremità e ridurre il problema escludendo la prima e l'ultima posizione.

6. **Conteggio dei picchi locali.** Scrivere una funzione ricorsiva che conti quanti elementi di un array A sono *picchi locali*, ossia valori tali che $A[i] > A[i - 1]$ e $A[i] > A[i + 1]$.
Suggerimento (da considerare solo in caso di difficoltà): limitare la ricorsione agli indici interni dell'array e verificare la condizione di massimo relativo.
7. **Calcolo del valore medio ricorsivo.** Scrivere una funzione ricorsiva che calcoli la media aritmetica degli elementi di un array A di lunghezza n .
Suggerimento (da considerare solo in caso di difficoltà): restituire la media come combinazione della somma parziale e del numero di elementi elaborati.
8. **Verifica della presenza di una sottosequenza crescente.** Scrivere una funzione ricorsiva che verifichi se all'interno di un array A esiste una sequenza di almeno tre elementi consecutivi in ordine strettamente crescente. La funzione deve restituire `true` o `false`.
Suggerimento (da considerare solo in caso di difficoltà): mantenere un contatore che si incrementa finché la sequenza è crescente e si azzerà in caso contrario.
9. **Conteggio dei valori univoci.** Scrivere una funzione ricorsiva che conti quanti elementi di un array A non si ripetono. Un elemento è detto univoco se compare una sola volta nell'array.
Suggerimento (da considerare solo in caso di difficoltà): confrontare ogni elemento con i restanti e proseguire riducendo l'array.
10. **Prodotto alternato degli elementi.** Scrivere una funzione ricorsiva che calcoli il prodotto alternato degli elementi di un array A , moltiplicando il primo, dividendo per il secondo, moltiplicando per il terzo, e così via.
Suggerimento (da considerare solo in caso di difficoltà): applicare a ciascun elemento un segno alternato $(-1)^i$ nel calcolo logaritmico o equivalente.

Esercizi di ricorsione su stringhe

1. **Conteggio dei caratteri.** Scrivere una funzione ricorsiva che, data una stringa S , restituisca il numero totale dei suoi caratteri.
Suggerimento (da considerare solo in caso di difficoltà): ridurre la lunghezza della stringa di un carattere a ogni passo e incrementare un contatore.
2. **Conteggio di una lettera specifica.** Scrivere una funzione ricorsiva che conti quante volte un carattere c compare all'interno di una stringa S .
Suggerimento (da considerare solo in caso di difficoltà): confrontare il primo carattere di S con c e proseguire sul resto della stringa.

3. **Verifica di palindromo.** Scrivere una funzione ricorsiva che determini se una stringa S è palindroma, cioè se può leggersi allo stesso modo da sinistra a destra e da destra a sinistra. La funzione deve restituire **true** o **false**.
Suggerimento (da considerare solo in caso di difficoltà): confrontare i caratteri alle estremità e ridurre la stringa escludendo il primo e l'ultimo carattere.
4. **Rimozione di un carattere.** Scrivere una funzione ricorsiva che elimini tutte le occorrenze di un carattere c da una stringa S e restituisca la nuova stringa risultante.
Suggerimento (da considerare solo in caso di difficoltà): costruire la nuova stringa aggiungendo i caratteri diversi da c e proseguire sulla parte rimanente.
5. **Conteggio delle vocali.** Scrivere una funzione ricorsiva che conti quante vocali (a, e, i, o, u) contiene una stringa S .
Suggerimento (da considerare solo in caso di difficoltà): controllare se il primo carattere appartiene all'insieme delle vocali e proseguire sul resto della stringa.
6. **Inversione di una stringa.** Scrivere una funzione ricorsiva che restituisca la stringa S invertita.
Suggerimento (da considerare solo in caso di difficoltà): spostare il primo carattere in fondo alla stringa e proseguire sull'insieme dei restanti.
7. **Conteggio delle maiuscole.** Scrivere una funzione ricorsiva che conti quanti caratteri maiuscoli contiene una stringa S .
Suggerimento (da considerare solo in caso di difficoltà): utilizzare il confronto con gli intervalli di codici ASCII o funzioni di libreria equivalenti.
8. **Verifica della presenza di una sottostringa.** Scrivere una funzione ricorsiva che determini se una stringa T è contenuta all'interno di una stringa S . La funzione deve restituire **true** o **false**.
Suggerimento (da considerare solo in caso di difficoltà): confrontare progressivamente il prefisso di S con T e scorrere ricorsivamente di un carattere.
9. **Rimozione delle cifre numeriche.** Scrivere una funzione ricorsiva che elimini da una stringa S tutti i caratteri numerici (0–9). La funzione deve restituire la stringa risultante.
Suggerimento (da considerare solo in caso di difficoltà): verificare a ogni passo se il primo carattere è una cifra numerica e, in caso contrario, mantenerlo nella costruzione della nuova stringa.
10. **Conteggio delle parole.** Scrivere una funzione ricorsiva che calcoli quante parole contiene una stringa S , assumendo che le parole siano separate da uno spazio singolo.
Suggerimento (da considerare solo in caso di difficoltà): incrementare il contatore quando si incontra una transizione da spazio a carattere non vuoto.

Esercizi di ricorsione su BST

1. **Conteggio dei nodi.** Scrivere una funzione ricorsiva che restituisca il numero totale di nodi presenti in un albero binario di ricerca.
Suggerimento (da considerare solo in caso di difficoltà): la dimensione di un albero è pari a 1 più la somma delle dimensioni dei due sottoalberi.
2. **Somma dei valori memorizzati.** Scrivere una funzione ricorsiva che calcoli la somma di tutti i valori numerici contenuti nei nodi di un BST.
Suggerimento (da considerare solo in caso di difficoltà): la somma complessiva si ottiene sommando il valore del nodo corrente con le somme dei due sottoalberi.
3. **Ricerca di un valore.** Scrivere una funzione ricorsiva che determini se un certo valore x è presente nel BST. La funzione deve restituire `true` o `false`.
Suggerimento (da considerare solo in caso di difficoltà): confrontare x con il valore del nodo corrente e proseguire a sinistra o a destra in base al risultato del confronto.
4. **Calcolo dell'altezza.** Scrivere una funzione ricorsiva che calcoli l'altezza di un BST, cioè la lunghezza del cammino più lungo dalla radice a una foglia.
Suggerimento (da considerare solo in caso di difficoltà): l'altezza è pari a 1 più il massimo tra le altezze dei due sottoalberi.
5. **Conteggio delle foglie.** Scrivere una funzione ricorsiva che determini quanti nodi foglia (senza figli) contiene il BST.
Suggerimento (da considerare solo in caso di difficoltà): un nodo è foglia se entrambi i sottoalberi sono nulli.
6. **Verifica di BST valido.** Scrivere una funzione ricorsiva che verifichi se un albero binario generico rispetta le proprietà di un BST. La funzione deve restituire `true` se per ogni nodo v tutti i valori nel sottoalbero sinistro sono minori di v , e tutti quelli nel sottoalbero destro sono maggiori.
Suggerimento (da considerare solo in caso di difficoltà): mantenere intervalli di validità (min, max) aggiornati a ogni chiamata.
7. **Ricerca del valore minimo.** Scrivere una funzione ricorsiva che restituisca il valore minimo memorizzato nel BST.
Suggerimento (da considerare solo in caso di difficoltà): in un BST il minimo si trova nel nodo più a sinistra.
8. **Ricerca del valore massimo.** Scrivere una funzione ricorsiva che restituisca il valore massimo memorizzato nel BST.
Suggerimento (da considerare solo in caso di difficoltà): in un BST il massimo si trova nel nodo più a destra.

9. **Conteggio dei nodi con un solo figlio.** Scrivere una funzione ricorsiva che conti quanti nodi del BST hanno esattamente un figlio non nullo.
Suggerimento (da considerare solo in caso di difficoltà): un nodo ha un solo figlio se uno dei due sottoalberi è nullo e l'altro no.
10. **Verifica della presenza di un percorso somma.** Scrivere una funzione ricorsiva che verifichi se esiste un cammino dalla radice a una foglia la cui somma dei valori dei nodi sia pari a un valore dato k . La funzione deve restituire `true` o `false`.
Suggerimento (da considerare solo in caso di difficoltà): a ogni passo sottrarre dal valore k il valore del nodo corrente e verificare la condizione al raggiungimento di una foglia.
11. **Somma dei nodi di un livello specifico.** Scrivere una funzione ricorsiva che calcoli la somma dei valori dei nodi presenti a un livello L specificato. Il livello della radice è 0.
Suggerimento (da considerare solo in caso di difficoltà): decrementare L a ogni passo fino a raggiungere il livello richiesto.
12. **Verifica di struttura identica.** Scrivere una funzione ricorsiva che determini se due BST hanno la stessa struttura (indipendentemente dai valori contenuti). La funzione deve restituire `true` se i due alberi hanno nodi disposti nello stesso modo.
Suggerimento (da considerare solo in caso di difficoltà): confrontare simultaneamente i due alberi visitando in parallelo i nodi corrispondenti.
13. **Somma dei valori compresi in un intervallo.** Scrivere una funzione ricorsiva che calcoli la somma dei valori dei nodi di un BST compresi tra due estremi a e b .
Suggerimento (da considerare solo in caso di difficoltà): sfruttare le proprietà del BST per evitare di visitare rami non necessari.
14. **Verifica di bilanciamento.** Scrivere una funzione ricorsiva che verifichi se un BST è bilanciato, ossia se per ogni nodo la differenza tra le altezze dei due sottoalberi non supera 1. La funzione deve restituire `true/false`.
Suggerimento (da considerare solo in caso di difficoltà): calcolare l'altezza dei sottoalberi durante la discesa e interrompere anticipatamente in caso di squilibrio.
15. **Copia del BST.** Scrivere una funzione ricorsiva che costruisca e restituisca una copia indipendente di un BST dato.
Suggerimento (da considerare solo in caso di difficoltà): creare un nuovo nodo con lo stesso valore del nodo corrente e ricopiare ricorsivamente i due sottoalberi.

Esercizi di ricorsione avanzati

1. **Conteggio delle inversioni in un array.** Dato un array A di lunghezza n , calcolare ricorsivamente il numero di coppie (i, j) con $i < j$ e $A[i] > A[j]$.
Suggerimento (da considerare solo in caso di difficoltà): integrare il conteggio nella fase di unione di un divide-et-impera stile merge.

2. **Selezione dell' k -esimo elemento (Quickselect ricorsivo).** Dato un array A e un indice k ($0 \leq k < n$), progettare una procedura ricorsiva che restituisca l'elemento di ordine k secondo l'ordinamento crescente.
Suggerimento (da considerare solo in caso di difficoltà): partizionare rispetto a un pivot e ricorrere solo nel sottoarray che contiene l'indice k .
3. **Partizionamento palindromico minimo di una stringa.** Data una stringa S , determinare ricorsivamente il minimo numero di tagli per suddividerla in sottostringhe palindrome.
Suggerimento (da considerare solo in caso di difficoltà): usare due indici per i confini e memorizzare sottoproblemi ripetuti.
4. **Corrispondenza con espressioni regolari semplificate.** Data una stringa S e un pattern P che può contenere i simboli speciali $.$ (un carattere qualsiasi) e $*$ (zero o più ripetizioni del precedente), stabilire ricorsivamente se P corrisponde interamente a S .
Suggerimento (da considerare solo in caso di difficoltà): implementare i casi $*$ consumando $0, 1, \dots$ caratteri compatibili e sfruttare pruning quando il match è impossibile.
5. **Sottosequenza palindroma massima (LPS).** Data una stringa S , calcolare ricorsivamente la lunghezza della più lunga sottosequenza palindroma.
Suggerimento (da considerare solo in caso di difficoltà): due indici i, j con ricorrenza che confronta $S[i]$ e $S[j]$ e memorizza i risultati.
6. **Numero di partizioni intere.** Per un intero $n \geq 1$, calcolare ricorsivamente quante partizioni di n esistono in parti non crescenti (ordine delle parti irrilevante).
Suggerimento (da considerare solo in caso di difficoltà): usare stato (n, m) con m massimo addendo consentito, riducendo n e/o m .
7. **Conteggio di sottoinsiemi con somma esatta.** Dato un array di interi A e un valore T , determinare ricorsivamente quanti sottoinsiemi di A sommano esattamente a T .
Suggerimento (da considerare solo in caso di difficoltà): scelte include/esclude sull'elemento corrente; pruning quando T diventa impossibile.
8. **Conteggio di cammini con somma esatta in un BST.** Dato un BST con valori interi e un intero K , contare ricorsivamente i cammini dalla radice alle foglie la cui somma è esattamente K .
Suggerimento (da considerare solo in caso di difficoltà): sottrarre il valore del nodo corrente da K e sommare i conteggi dei sottoalberi.
9. **Antenato comune più basso (LCA) in un BST.** Dati un BST e due chiavi distinte x, y presenti nell'albero, progettare una procedura ricorsiva che restituisca il loro LCA.

Suggerimento (da considerare solo in caso di difficoltà): usare il confronto con il valore del nodo corrente per scegliere sinistra/destra o fermarsi.

10. **Equivalenza con ribaltamento di sottoalberi (flip-equivalence).** Date le radici di due BST, determinare ricorsivamente se sono equivalenti a meno di scambi tra figli sinistro e destro in nodi arbitrari.

Suggerimento (da considerare solo in caso di difficoltà): confrontare ricorsivamente sia la coppia $(L \leftrightarrow L, R \leftrightarrow R)$ sia $(L \leftrightarrow R, R \leftrightarrow L)$.

11. **Ricostruzione di un BST dal preorder.** Data una sequenza di visite in preorder di un BST con chiavi distinte, ricostruire ricorsivamente l'albero.

Suggerimento (da considerare solo in caso di difficoltà): passare limiti (min, max) e un indice globale/di riferimento per consumare i nodi validi.

12. **Conteggio delle coppie con differenza limitata.** Dato un array A di lunghezza n e un intero $D \geq 0$, contare ricorsivamente quante coppie (i, j) con $i < j$ soddisfano $|A[i] - A[j]| \leq D$.

Suggerimento (da considerare solo in caso di difficoltà): schema divide-et-impera con unione che conta coppie inter-partizione in modo ordinato.

13. **Minimo numero di rimozioni per rendere una stringa palindroma.** Data una stringa S , calcolare ricorsivamente il minimo numero di caratteri da rimuovere per ottenere un palindromo.

Suggerimento (da considerare solo in caso di difficoltà): ricorrenza su due indici i, j ; collegare alla LPS per derivare il risultato.

14. **Partizioni con limiti di cardinalità.** Dato un intero n e un limite k , contare ricorsivamente in quanti modi n può essere scritto come somma di *al più* k interi positivi non crescenti.

Suggerimento (da considerare solo in caso di difficoltà): estendere lo stato a (n, m, k) con massimo addendo m e parti residue k .